Informe de práctica de la semana 2

Encapsulamiento e invariantes

UNAP · Ingeniería de Sistemas · SIS206 POO II · 2026 II
S2-001 · 19 de octubre de 2026 · Informe revisado

1 Propósito y actividades

Proteger el estado de Curso y Docente mediante propiedades y operaciones que preservan invariantes, y trasladar el mismo criterio a las tres clases iniciales del hotel.

Las plantillas dejan pendientes los setters de código, créditos y cupos. Además, limitar los cupos a 1 a 50 no basta: una reducción nunca debe situar el máximo por debajo de los inscritos. La edad y los totales del docente deben calcularse desde sus datos actuales, sin duplicar valores derivados.

Actividades desarrolladas

Actividad	Archivo o componente	Resultado
1 Curso	python/solucion.py	Código validado; créditos 1 a 6; cupos 1 a 50; inscritos protegidos.
2 Docente	python/solucion.py	DNI de ocho dígitos, correo UNAP, edad y totales calculados; máximo tres cursos.
3 C++	cpp/solucion.hpp	Setters validados y consultas const; mismo caso de demostración.
4 Equivalencias	ACTIVIDADES_RESUELTAS.md	Tabla completa y explicación del uso de const.
5 Hotel	python/hotel_base.py y cpp/hotel_base.hpp	Huesped, Habitacion y Reserva con invariantes y operaciones de negocio.

Caso explicado

Un curso dispone de tres cupos y ya tiene dos inscritos. Reducir su capacidad a uno produciría un estado imposible. El setter rechaza el cambio antes de asignarlo; por ello se conservan los tres cupos y queda una plaza libre. La misma regla protege a una habitación cuando se intenta reducir su capacidad por debajo de sus ocupantes.

Conceptos para interpretar el caso

Una invariante es una condición que debe mantenerse durante la vida del objeto. Una propiedad calculada obtiene su valor de otros datos; por ejemplo, cupos_disponibles es la diferencia entre capacidad e inscritos y no necesita almacenarse por separado.

La solución y sus pruebas se encuentran en desarrollo. Las respuestas completas y el notebook amplían los ejemplos de este informe.

2 Desarrollo en Python y C++

Se siguió el patrón explícito de las dos plantillas: entre dos y cuatro letras mayúsculas y tres dígitos ASCII. El enunciado breve menciona SIS seguido de dígitos; esta diferencia se declara para que el contrato elegido pueda revisarse. SIS206 se acepta en ambas interpretaciones.

Los constructores usan las validaciones de sus setters. Los cambios se comprueban antes de asignar, de modo que un error conserva el valor anterior. La colección de cursos del docente se consulta como tupla en Python y se comparte de manera controlada en C++ para reflejar las modificaciones de cada curso.

La edad se calcula con fecha actual; edad_en permite comprobar cumpleaños con una referencia fija, 18 de octubre de 2026, sin depender del día de ejecución. nombre_completo, cupos_disponibles y los totales no ofrecen setter.

Equivalencias de encapsulamiento

Concepto	Python	C++
Privado	__atributo modifica el nombre	private restringe el acceso
Protegido	_atributo es una convención	protected es una restricción del lenguaje
Getter	@property sin setter	Método de consulta const
Setter	@atributo.setter con validación	setAtributo con comprobación y excepción
Propiedad calculada	@property que calcula al consultar	Getter const que calcula al consultar
No modificar	No existe un const equivalente	Calificador const para la operación
Validación	raise ValueError	throw std::invalid_argument

Cómo seguir la implementación

En Python, la asignación c.cupos_maximo = 1 activa el setter decorado con @cupos_maximo.setter. En C++, c.setCuposMaximo(1) realiza la misma comprobación. Ambas operaciones validan antes de guardar. Las consultas se exponen mediante @property o getters const; las propiedades calculadas se recomputan al consultarlas.

3 Verificación y resultados

Las pruebas registradas aprobaron 17 casos Python y 16 casos C++17. Se ejecutaron en Python 3.12.14 y g++ 10.3.0. Las demostraciones de ambos lenguajes producen la misma salida.

Comprobación	Resultado	Registro
Pruebas Python	17 aprobadas	evidencias/pruebas_python.txt
Pruebas C++17	16 aprobadas	evidencias/pruebas_cpp.txt
Compilación y demostraciones	Sin advertencias y salidas iguales	evidencias/verificacion.json

Pytest ejecutó los mismos casos que unittest; sus resultados no se suman como pruebas diferentes. Las compilaciones usaron -Wall, -Wextra, -Wpedantic y -Werror.

Casos representativos comprobados

Caso	Resultado observado	Qué demuestra
Créditos en los límites 1 y 6	Aceptados	Los extremos válidos pertenecen al intervalo.
Dos inscritos y reducción de tres cupos a uno	Cambio rechazado; capacidad sigue en 3	El error no deja el objeto parcialmente modificado.
Cumpleaños el 14 de mayo de 1980	45 años el 13 de mayo de 2026 y 46 el día 14	La edad considera si el cumpleaños ya ocurrió.
Asignar un cuarto curso o repetir uno	Asignación rechazada	Se preservan el máximo de tres y la ausencia de duplicados.

Las pruebas comprueban el valor calculado y el estado posterior a una operación inválida. Que una excepción se produzca no basta: también debe conservarse la información válida que el objeto tenía antes del intento.

Lectura de la salida

LIBRES=27 resulta de restar tres inscritos a treinta cupos. EDAD_REFERENCIA=46 corresponde a la fecha fija usada en la demostración. Las pruebas adicionales comprueban el día anterior al cumpleaños y el propio día del cumpleaños.

4 Aplicación al hotel y conclusiones

Huesped protege identificación, nombre y documento, calcula una etiqueta y valida la actualización de documento. Habitacion protege número, tipo, capacidad, tarifa y ocupación; calcula plazas libres y comprueba alojar. Reserva protege código y fechas, calcula noches y controla confirmar o anular. Los identificadores y las fechas se mantienen de solo lectura cuando no existe un cambio de negocio justificado.

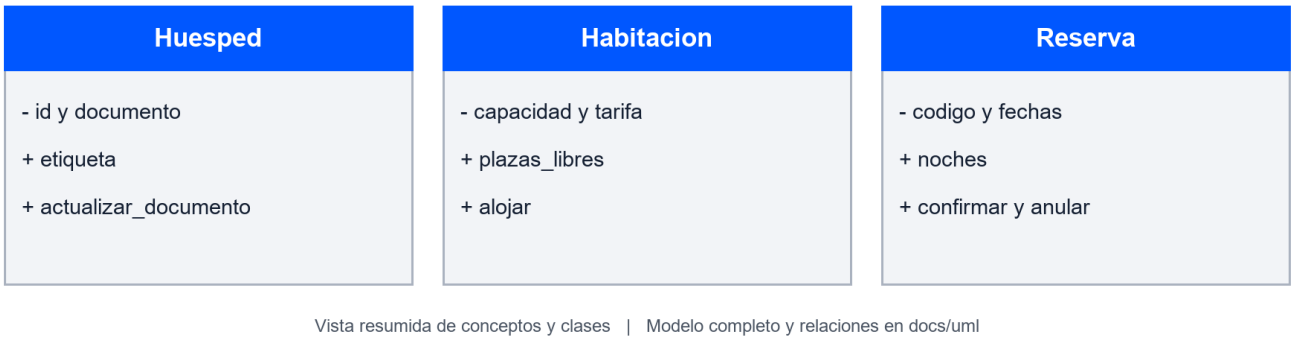


Figura 1 Esquema de los conceptos aplicados al hotel. El diagrama de clases completo se encuentra en docs/uml.

Conclusión de la práctica

El encapsulamiento quedó ligado a reglas de negocio observables. Las propiedades de Curso y Docente evitan duplicar datos derivados, y las operaciones del hotel preservan capacidad, fechas y estado. La regla decisiva es validar antes de modificar.

Para comprobar la comprensión, seguir el caso explicado en los dos lenguajes e identificar la validación, el cambio de estado y la prueba que lo verifica. El anexo ACTIVIDADES_RESUELTAS.md desarrolla las preguntas de reflexión.

Transparencia y datos de entrega

Se utilizó Codex para analizar el material, desarrollar y verificar el código y preparar los documentos. Antes de entregar, completar los datos del estudiante y GitHub. La revisión de pares y la sustentación requieren participación humana.

Referencias

Fowler, M. (2018). *Refactoring Improving the Design of Existing Code* (2.ª ed.). Addison Wesley. https://martinfowler.com/books/refactoring.html

Object Management Group. (2017). *OMG Unified Modeling Language Version 2.5.1*. https://www.omg.org/spec/UML/2.5.1/

Python Software Foundation. (s. f.). *abc — Abstract Base Classes*. Python 3.12 documentation. https://docs.python.org/3.12/library/abc.html

Zanabria Gálvez, A. H. (2026). *Guía de práctica de laboratorio de la semana 2*. SIS206 POO II, Universidad Nacional del Altiplano. Material del curso.